Projet Pétale : antisèche

Le piston rejoue le flux de sève de l'arbre n°9 : plus le flux est fort, plus il monte.

1. La chaîne complète

Serveur de Julien (données forêt téléchargées) → Python (VS Code) → câble USB → ESP32 → piston + LED

- **Python** télécharge les données, calcule les 12 hauteurs et envoie les ordres.
- **L'ESP32** exécute : il fait bouger le servo et pulser les LED.

2. Rôle des fichiers

Fichier	Rôle
config.py	Tous les réglages : adresses des données, 12 états, 60 s, angles, port COM3.
donnees_foret.py	Télécharge les mesures de l'arbre n°9 et les découpe en 12 hauteurs (0° à 180°).
liaison_esp32.py	Ouvre la connexion USB et envoie les ordres à l'ESP32 (POS:90, STOP…).
main.py	Le programme à lancer : retour en bas, puis les 12 états dans l'ordre (≈ 1 min 10).
petale_esp32.ino	Programme de l'ESP32 : reçoit les ordres, bouge le servo, fait pulser les LED.

3. Pourquoi ces données

Source	Contenu	Utilisée ?
2023-07 à 2023-10.json (serveur de Julien)	Toutes les mesures du 18/07 au 12/10/2023, une toutes les 30 min (4 112 valeurs).	OUI : on peut parcourir toute la colonne d'un coup.
live.json	Une seule valeur : la mesure du moment, changée toutes les 30 min.	NON : impossible d'en tirer 12 états en 1 minute.
Excel forêt (PC)	SF_diff journalier 2022-2024, avec des valeurs corrompues (#NOM?).	NON : la consigne est d'utiliser la forêt en ligne.

- Fichiers mensuels et live.json = **mêmes valeurs, même capteur** ; live.json n'en montre qu'une à la fois.
- Chaque état = une période d'environ une semaine. Valeur = écart jour/nuit du capteur (= SF_diff), médiane de la période.
- Période la plus faible → piston en bas ; la plus forte → en haut. Valeurs invalides ignorées, rien n'est inventé.

3 bis. Comment les 12 états sont calculés (donnees_foret.py)

1. **Télécharger :** les 4 fichiers du dossier data de Julien. On garde SF_tree_09_Avg (arbre n°9), on ignore les valeurs invalides → 4 112 mesures triées dans l'ordre.
2. **Une valeur par jour :** écart entre la mesure la plus haute et la plus basse de la journée (= SF_diff). Jours incomplets écartés → ≈ 86 jours.
3. **12 groupes dans l'ordre :** les 86 jours sont répartis en 12 périodes successives d'≈ 7 jours.
4. **Médiane de chaque groupe (pas la moyenne) :** la valeur du milieu des 7 jours. Elle ignore le pic aberrant du capteur du 20/07.
5. **Hauteur :** groupe le plus faible = 0° (bas), le plus fort = 180° (haut), les autres proportionnellement entre les deux.

En une phrase : *on télécharge l'historique de l'arbre n°9, on calcule le flux de chaque jour, on regroupe les jours en 12 semaines dans l'ordre et on prend la médiane de chaque semaine pour fixer la hauteur du piston.*

4. VS Code ou IDE Arduino ?

	VS Code (Python)	IDE Arduino (C++)
Tourne sur	Le PC	L'ESP32 (après téléversement)
Rôle	Le cerveau : récupère les données, décide des hauteurs	Le corps : exécute les ordres, gère servo et LED
Quand	À chaque démo : python src/main.py	Une seule fois, pour téléverser le programme

- L'IDE Arduino sert seulement à **écrire et téléverser** le programme dans l'ESP32. Ensuite, on le ferme.
- Python fait le calcul sur PC (plus simple, respecte la consigne Python/VS Code). L'ESP32 ne fait que bouger.
- **Règle d'or :** un seul programme à la fois sur COM3. Fermer l'IDE Arduino avant de lancer Python.

5. Branchements

Élément	Fil	Borne ESP32	Rôle
Servo (piston)	Marron	GND	Masse (−)
Servo (piston)	Rouge	VIN	Alimentation 5V (+)
Servo (piston)	Orange	D18	Signal
Ruban LED	Rouge	VIN (avec le servo)	Alimentation 5V (+)
Ruban LED	Blanc	GND (l'autre borne)	Masse (−)
Ruban LED	Vert	D4	Signal

6. Lancer la démo

1. Brancher l'ESP32 en USB (programme déjà téléversé).
2. Vérifier que l'IDE Arduino est fermé.
3. **Dans VS Code :** `python src/main.py`
4. Le piston redescend, puis parcourt les 12 états (≈ 1 min 10) ; les LED suivent la hauteur.

7. Questions probables

Question	Réponse
C'est du live ?	Non. C'est un historique fixe (18/07 → 12/10/2023) téléchargé en ligne. Seul live.json change toutes les 30 min, et c'est un rejeu de 2023 : il n'y a plus de capteur actif.
Pourquoi pas live.json ?	Il ne contient qu'une valeur à la fois. Pour 12 états, il faudrait attendre 12 × 30 min = 6 h.
Pourquoi l'écart jour/nuit ?	C'est la définition du SF_diff : plus l'écart du capteur est grand, plus la sève circule.
Pourquoi la médiane ?	Un pic aberrant du capteur le 20/07 aurait propulsé le 1er état tout en haut avec une moyenne. La médiane l'ignore.
Pourquoi 12 états ?	Lisible en ≈ 1 min : ≈ 5 s par hauteur pour voir le niveau et lire l'info dans le terminal.
Et les valeurs invalides ?	Ignorées, jamais inventées (pas d'interpolation).
Pourquoi Python ET Arduino ?	Python calcule sur le PC (cerveau), l'ESP32 exécute (corps). L'IDE Arduino sert juste à téléverser.
Pourquoi le retour en bas ?	Chaque démo part du même point, quelle que soit la position précédente.
Que font les LED ?	Elles pulsent selon la hauteur : en bas miel, lent et doux ; en haut marron, rapide et intense.